Stack and Queues

1. Implement Stack using Array

You need to implement a stack data structure using an array.
A stack follows the **Last-In-First-Out (LIFO)** principle, meaning the last element added is the first one removed.

The stack must support these operations:

- push(x): insert an element into the stack
- pop(): remove and return the top element
- top(): return the top element without removing it
- isEmpty(): check whether the stack is empty

Example:
If we push 5, then push 10, the top element becomes 10.
If we pop, 10 is removed.
After that, the stack is not empty because 5 is still present.

Approach 1: Array Implementation

Algorithm
Create an array with a fixed size to store stack elements.
Maintain an integer variable `topIndex` to track the index of the top element.
Initialize `topIndex` as -1 to indicate an empty stack.

Push operation:

- Check if the stack is full.
- Increase `topIndex` and insert the element at that index.

Pop operation:

- Check if the stack is empty.
- Return the element at `topIndex` and decrease `topIndex`.

Top operation:

- Check if the stack is empty.
- Return the element at `topIndex` without removing it.

isEmpty operation:

- If `topIndex` is -1, the stack is empty.

Code

```
#include <bits/stdc++.h>
using namespace std;

class ArrayStack {
private:
    int* stackArray;
    int capacity;
    int topIndex;

public:
    ArrayStack(int size = 1000) {
        capacity = size;
        stackArray = new int[capacity];
        topIndex = -1;
    }

    ~ArrayStack() {
        delete[] stackArray;
    }

    void push(int x) {
        if (topIndex >= capacity - 1) {
            cout << "Stack overflow" << endl;
            return;
        }
        stackArray[++topIndex] = x;
    }
```

```
    int pop() {
        if (isEmpty()) {
            cout << "Stack is empty" << endl;
            return -1;
        }
        return stackArray[topIndex--];
    }

    int top() {
        if (isEmpty()) {
            cout << "Stack is empty" << endl;
            return -1;
        }
        return stackArray[topIndex];
    }

    bool isEmpty() {
        return topIndex == -1;
    }
};

int main() {
    ArrayStack stack;
    stack.push(5);
    stack.push(10);
    cout << stack.top() << " ";
    cout << stack.pop() << " ";
    cout << (stack.isEmpty() ? "true" : "false");
    return 0;
}
```

Complexity Analysis

Time Complexity: O(1) for push, pop, top, and isEmpty operations.
Space Complexity: O(N), where N is the capacity of the stack array.

2. Implement Queue Using Array

You need to implement a queue using an array.
A queue follows the **First-In-First-Out (FIFO)** principle, meaning the element inserted first is removed first.

The queue must support these operations:

- push(x): add an element to the end of the queue
- pop(): remove and return the front element
- peek(): return the front element without removing it
- isEmpty(): check whether the queue is empty

Example:
If we push 5, then push 10, the front element is 5.
If we pop, 5 is removed.
After that, the queue is not empty because 10 is still present.

Approach 1: Circular Queue using Array

Algorithm
Create an array of fixed size to store queue elements.
Maintain four variables:

- start: index of the front element
- end: index of the last element
- currSize: current number of elements
- maxSize: maximum capacity of the queue

Push operation:

- Check if the queue is full using currSize.

- If the queue is empty, initialize start and end to 0.
- Otherwise, move end forward using circular indexing.
- Insert the element and increase currSize.

Pop operation:

- Check if the queue is empty.
- Store the element at start.
- If only one element exists, reset start and end to -1.
- Otherwise, move start forward using circular indexing.
- Decrease currSize and return the removed element.

Peek operation:

- Return the element at start if the queue is not empty.

isEmpty operation:

- If currSize is 0, the queue is empty.

Code

```
#include <bits/stdc++.h>
using namespace std;

class ArrayQueue {
    int* arr;
    int start,end;
    int currSize,maxSize;

public:
    ArrayQueue() {
        maxSize=10;
        arr=new int[maxSize];
```

```
    start=-1;
    end=-1;
    currSize=0;
}

void push(int x) {
    if(currSize==maxSize) {
        cout<<"Queue is full"<<endl;
        return;
    }
    if(end==-1) {
        start=0;
        end=0;
    } else {
        end=(end+1)%maxSize;
    }
    arr[end]=x;
    currSize++;
}

int pop() {
    if(isEmpty()) {
        cout<<"Queue is empty"<<endl;
        return -1;
    }
    int val=arr[start];
    if(currSize==1) {
        start=-1;
        end=-1;
    } else {
        start=(start+1)%maxSize;
    }
    currSize--;
    return val;
}

int peek() {
    if(isEmpty()) {
```

```
            cout<<"Queue is empty"<<endl;
            return -1;
        }
        return arr[start];
    }

    bool isEmpty() {
        return currSize==0;
    }
};

int main() {
    ArrayQueue queue;
    queue.push(5);
    queue.push(10);
    cout<<queue.peek()<<" ";
    cout<<queue.pop()<<" ";
    cout<<(queue.isEmpty()?"true":"false");
    return 0;
}
```

Complexity Analysis

Time Complexity: O(1) for push, pop, peek, and isEmpty operations.

Space Complexity: O(N), where N is the fixed size of the array used for the queue.

3. Implement Stack using single Queue

You need to implement a stack using only one queue.

A stack follows the **Last-In-First-Out (LIFO)** rule, while a queue follows **First-In-First-Out (FIFO)**.

The goal is to use queue operations in such a way that stack behavior is achieved.

The stack must support:

- push(x): insert an element
- pop(): remove and return the top element
- top(): return the top element without removing it
- isEmpty(): check if the stack is empty

Example:
If we push 4, then push 8, the top element becomes 8.
If we pop, 8 is removed.
After that, the top element becomes 4.

Approach 1: Using One Queue

Algorithm
Use a single queue to store all elements.
For push(x):

- First, insert x into the queue.
- Let the current size of the queue before insertion be s.
- Rotate the queue s times by removing the front element and pushing it to the back.
- This brings the newly added element to the front, making it act like the top of a stack.

For pop():

- Remove and return the front element of the queue. This represents the stack's top.

For top():

- Return the front element of the queue without removing it.

For isEmpty():

- If the queue is empty, return true, otherwise false.

Code

```
#include <bits/stdc++.h>
using namespace std;

class QueueStack {
    queue<int> q;

public:
    void push(int x) {
        int s = q.size();
        q.push(x);
        for(int i = 0; i < s; i++) {
            q.push(q.front());
            q.pop();
        }
    }

    int pop() {
        int val = q.front();
        q.pop();
        return val;
    }

    int top() {
        return q.front();
    }

    bool isEmpty() {
        return q.empty();
    }
};

int main() {
    QueueStack st;
```

```
    st.push(4);
    st.push(8);
    cout << st.pop() << " ";
    cout << st.top() << " ";
    cout << (st.isEmpty() ? "true" : "false");
    return 0;
}
```

Complexity Analysis
Time Complexity:

- push(): O(n), because existing elements are rotated.
- pop(): O(1)
- top(): O(1)
- isEmpty(): O(1)

Space Complexity: O(n), where n is the number of elements stored in the queue.

4. Implement Queue using Stack

You need to implement a queue using stacks.
A queue follows the **First-In-First-Out (FIFO)** principle, but stacks follow **Last-In-First-Out (LIFO)**.
Using stack operations carefully, queue behavior can be achieved.

The queue must support:

- push(x): add element to the end of the queue
- pop(): remove and return the front element

- peek(): return the front element without removing it
- isEmpty(): check whether the queue is empty

Example:
If we push 4, then push 8, the front element is 4.
If we pop, 4 is removed.
After that, peek returns 8.

Approach 1: Using Two Stacks (Push is O(N))

Algorithm
Maintain two stacks: st1 and st2.

Push operation:

- Move all elements from st1 to st2.
- Push the new element into st1.
- Move all elements back from st2 to st1.
- This ensures the front of the queue is always on top of st1.

Pop operation:

- Remove and return the top element of st1.

Peek operation:

- Return the top element of st1 without removing it.

isEmpty operation:

- Check if st1 is empty.

Code

```
#include <bits/stdc++.h>
```

```
using namespace std;

class StackQueue {
    stack<int> st1, st2;

public:
    void push(int x) {
        while(!st1.empty()){
            st2.push(st1.top());
            st1.pop();
        }
        st1.push(x);
        while(!st2.empty()){
            st1.push(st2.top());
            st2.pop();
        }
    }

    int pop() {
        if(st1.empty()) return -1;
        int val = st1.top();
        st1.pop();
        return val;
    }

    int peek() {
        if(st1.empty()) return -1;
        return st1.top();
    }

    bool isEmpty() {
        return st1.empty();
    }
};

int main() {
    StackQueue q;
    q.push(4);
```

```
    q.push(8);
    cout << q.pop() << " ";
    cout << q.peek() << " ";
    cout << (q.isEmpty() ? "true" : "false");
    return 0;
}
```

Complexity Analysis
Time Complexity:

- push(): O(N)
- pop(): O(1)
- peek(): O(1)
- isEmpty(): O(1)

Space Complexity: O(N), for storing elements in stacks.

Approach 2: Using Two Stacks (Push is O(1))

Algorithm
Maintain two stacks: input and output.

Push operation:

- Simply push the element into input stack.

Pop operation:

- If output stack is empty, move all elements from input to output.
- Pop and return the top of output stack.

Peek operation:

- If output stack is empty, move all elements from input to output.

- Return the top of output stack.

isEmpty operation:

- Check if both stacks are empty.

Code

```
#include <bits/stdc++.h>
using namespace std;

class StackQueue {
    stack<int> input, output;

public:
    void push(int x) {
        input.push(x);
    }

    int pop() {
        if(output.empty()){
            while(!input.empty()){
                output.push(input.top());
                input.pop();
            }
        }
        if(output.empty()) return -1;
        int val = output.top();
        output.pop();
        return val;
    }

    int peek() {
        if(output.empty()){
            while(!input.empty()){
                output.push(input.top());
                input.pop();
            }
```

```
        }
        if(output.empty()) return -1;
        return output.top();
    }

    bool isEmpty() {
        return input.empty() && output.empty();
    }
};

int main() {
    StackQueue q;
    q.push(4);
    q.push(8);
    cout << q.pop() << " ";
    cout << q.peek() << " ";
    cout << (q.isEmpty() ? "true" : "false");
    return 0;
}
```

Complexity Analysis

Time Complexity:

- push(): O(1)
- pop(): O(N) in worst case, O(1) amortized
- peek(): O(N) in worst case, O(1) amortized
- isEmpty(): O(1)

Space Complexity: O(N), for storing elements in two stacks.

5. Implement Stack using Linked List

You need to implement a stack using a singly linked list.
A stack follows the **Last-In-First-Out (LIFO)** principle, where the last inserted element is removed first.

The stack must support:

- push(x): insert an element on top of the stack
- pop(): remove and return the top element
- top(): return the top element without removing it
- isEmpty(): check whether the stack is empty

Example:
If we push 3, then push 7, the top becomes 7.
If we pop, 7 is removed.
After that, the top becomes 3.

Approach 1: Using Singly Linked List

Algorithm
Use a singly linked list where the head represents the top of the stack.

Push operation:

- Create a new node with value x.
- Point the new node's next to the current head.
- Update head to the new node.

Pop operation:

- If the stack is empty, return -1.
- Store the value of head.
- Move head to head->next.
- Delete the old head and return the stored value.

Top operation:

- If the stack is empty, return -1.
- Return the value stored at head.

isEmpty operation:

- If head is NULL, the stack is empty.

Code

```
#include <bits/stdc++.h>
using namespace std;

struct Node {
    int val;
    Node* next;
    Node(int x) {
        val = x;
        next = NULL;
    }
};

class LinkedListStack {
    Node* head;
    int size;

public:
    LinkedListStack() {
```

```
        head = NULL;
        size = 0;
    }

    void push(int x) {
        Node* node = new Node(x);
        node->next = head;
        head = node;
        size++;
    }

    int pop() {
        if (head == NULL) return -1;
        int val = head->val;
        Node* temp = head;
        head = head->next;
        delete temp;
        size--;
        return val;
    }

    int top() {
        if (head == NULL) return -1;
        return head->val;
    }

    bool isEmpty() {
        return size == 0;
    }
};

int main() {
    LinkedListStack st;
    st.push(3);
    st.push(7);
    cout << st.pop() << " ";
    cout << st.top() << " ";
    cout << (st.isEmpty() ? "true" : "false");
```

```
    return 0;
}
```

Complexity Analysis
Time Complexity: O(1) for push, pop, top, and isEmpty operations.
Space Complexity: O(N), where N is the number of elements stored in the stack.

6. Implement Queue using Linked List

You need to implement a queue using a singly linked list.
A queue follows the **First-In-First-Out (FIFO)** principle, meaning the element inserted first is removed first.

The queue must support:

- push(x): insert an element at the end of the queue
- pop(): remove and return the front element
- peek(): return the front element without removing it
- isEmpty(): check whether the queue is empty

Example:
If we push 3, then push 7, the front element is 3.
If we pop, 3 is removed.
After that, the queue is not empty because 7 is still present.

Approach 1: Using Singly Linked List

Algorithm
Use a singly linked list with two pointers:

- `start` points to the front of the queue

- end points to the rear of the queue

Push operation:

- Create a new node with value x.
- If the queue is empty, set both start and end to the new node.
- Otherwise, link the current end node to the new node and update end.

Pop operation:

- If the queue is empty, return -1.
- Store the value at start.
- Move start to start->next.
- Delete the old front node.
- If the queue becomes empty, end will naturally become NULL.

Peek operation:

- If the queue is empty, return -1.
- Return the value at start.

isEmpty operation:

- If start is NULL (or size is 0), the queue is empty.

Code

```
#include <bits/stdc++.h>
using namespace std;

struct Node {
    int val;
    Node* next;
```

```
    Node(int x) {
        val = x;
        next = NULL;
    }
};

class LinkedListQueue {
private:
    Node* start;
    Node* end;
    int size;

public:
    LinkedListQueue() {
        start = end = NULL;
        size = 0;
    }

    void push(int x) {
        Node* node = new Node(x);
        if(start == NULL) {
            start = end = node;
        } else {
            end->next = node;
            end = node;
        }
        size++;
    }

    int pop() {
        if(start == NULL) return -1;
        int val = start->val;
        Node* temp = start;
        start = start->next;
        delete temp;
        size--;
        return val;
    }
```

```
    int peek() {
        if(start == NULL) return -1;
        return start->val;
    }

    bool isEmpty() {
        return size == 0;
    }
};

int main() {
    LinkedListQueue q;
    q.push(3);
    q.push(7);
    cout << q.peek() << " ";
    cout << q.pop() << " ";
    cout << (q.isEmpty() ? "true" : "false");
    return 0;
}
```

Complexity Analysis
Time Complexity: O(1) for push, pop, peek, and isEmpty operations.
Space Complexity: O(N), where N is the number of elements stored in the queue.

7. Check for Balanced Parentheses

Given a string containing only the characters `(`, `)`, `{`, `}`, `[` and `]`, check whether the string is balanced.
A string is balanced if every opening bracket has a matching closing bracket of the same type and the brackets are closed in the correct order.

For example, `"()[{}()]"` is balanced because every opening bracket is correctly closed.
The string `"[( )"` is not balanced because `[` does not have a matching `]`.

Approach 1: Using Stack

Algorithm

We need to track the most recent opening bracket and match it with the current closing bracket. A stack follows Last-In-First-Out behavior, which perfectly fits this requirement.

Traverse the string character by character.
If the character is an opening bracket (, {, or [, push it onto the stack.
If the character is a closing bracket:

- If the stack is empty, there is no matching opening bracket, so return false.
- Otherwise, check the top of the stack. If it is not the matching opening bracket, return false.
- If it matches, pop the opening bracket and continue.

After processing the entire string, the stack must be empty.
If it is empty, all brackets were matched correctly, so return true. Otherwise, return false.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    bool isValid(string s) {
        stack<char> st;
        for(char ch : s) {
            if(ch == '(' || ch == '{' || ch == '[') {
                st.push(ch);
            } else {
                if(st.empty()) return false;
                char top = st.top();
                st.pop();
                if((ch == ')' && top == '(') ||
                   (ch == ']' && top == '[') ||
                   (ch == '}' && top == '{')) {
```

```
                continue;
            } else {
                return false;
            }
        }
    }
    return st.empty();
  }
};

int main() {
    Solution sol;
    string s = "()[{}()]";
    cout << (sol.isValid(s) ? "True" : "False");
    return 0;
}
```

Complexity Analysis

Time Complexity: O(N), where N is the length of the string, as each character is processed once.

Space Complexity: O(N), due to the stack storing opening brackets in the worst case.

8. Implement Min Stack : O(2N) and O(N) Space Complexity

You need to design a stack that supports normal stack operations along with retrieving the minimum element, all in constant time.

The stack must support:

- push(val): insert an element

- pop(): remove the top element
- top(): return the top element
- getMin(): return the minimum element present in the stack

Example:
If we push -2, 0, -3 into the stack, the minimum is -3.
After popping -3, the top becomes 0 and the minimum becomes -2.

Approach 1: Brute Force (Using Stack of Pairs)

In this approach, each element in the stack stores:

- the actual value
- the minimum value up to that point

So for every element, we always know the minimum till that element.

Algorithm
Use a stack of pairs (value, currentMinimum).

Push operation:

- If the stack is empty, push (value, value).
- Otherwise, compare value with the current minimum.
- Push (value, min(value, currentMinimum)).

Pop operation:

- Remove the top pair from the stack.

Top operation:

- Return the first element of the top pair.

GetMin operation:

- Return the second element of the top pair.

Code

```
#include <bits/stdc++.h>
using namespace std;

class MinStack {
private:
    stack<pair<int,int>> st;

public:
    MinStack() {}

    void push(int value) {
        if(st.empty()) {
            st.push({value,value});
        } else {
            int mini = min(value, st.top().second);
            st.push({value, mini});
        }
    }

    void pop() {
        if(!st.empty()) st.pop();
    }

    int top() {
        if(st.empty()) return -1;
        return st.top().first;
    }

    int getMin() {
        if(st.empty()) return -1;
        return st.top().second;
    }
};
```

```
int main() {
    MinStack s;
    s.push(-2);
    s.push(0);
    s.push(-3);
    cout << s.getMin() << " ";
    s.pop();
    cout << s.top() << " ";
    cout << s.getMin();
    return 0;
}
```

Complexity Analysis
Time Complexity: O(1) for push, pop, top, and getMin.
Space Complexity: O(2N) ≈ O(N), because each element stores an extra minimum value.

Approach 2: Optimal Approach (Single Stack with Encoding)

In this approach, only one stack is used along with a variable to track the current minimum.

The idea is to encode values smaller than the current minimum before pushing them.

Algorithm
Maintain:

- one stack
- one variable `mini` to store the current minimum

Push operation:

- If the stack is empty, push value and set mini = value.
- If value > mini, push value normally.
- If value <= mini, push a modified value (2*value - mini) and update mini to value.

Pop operation:

- If the top value is >= mini, pop normally.
- If the top value < mini, it means it was encoded.
    - Update mini = 2*mini - top.

Top operation:

- If top >= mini, return top.
- Otherwise, return mini.

GetMin operation:

- Return mini.

Main logic:

Encoded value hamesha current mini se chhoti hoti hai

Proof:

`encoded = 2*value - mini`

`Since value < mini`

`⇒ 2*value - mini < value`

`⇒ encoded < value < mini`

👉 Matlab:

- Stack me koi element **mini se chhota** dikhe
  ⇒ wo **real value nahi**, encoded value hai

Code

```
#include <bits/stdc++.h>
using namespace std;
```

```
class MinStack {
private:
    stack<int> st;
    int mini;

public:
    MinStack() {}

    void push(int value) {
        if(st.empty()) {
            mini = value;
            st.push(value);
        } else if(value > mini) {
            st.push(value);
        } else {
            st.push(2*value - mini);
            mini = value;
        }
    }

    void pop() {
        if(st.empty()) return;
        int x = st.top();
        st.pop();
        if(x < mini) {
            mini = 2*mini - x;
        }
    }

    int top() {
        if(st.empty()) return -1;
        int x = st.top();
        if(x >= mini) return x;
        return mini;
    }

    int getMin() {
```

```
        return mini;
    }
};

int main() {
    MinStack s;
    s.push(-2);
    s.push(0);
    s.push(-3);
    cout << s.getMin() << " ";
    s.pop();
    cout << s.top() << " ";
    cout << s.getMin();
    return 0;
}
```

Complexity Analysis
Time Complexity: O(1) for push, pop, top, and getMin.
Space Complexity: O(N), as only one stack is used without storing extra pairs.

🧪 Dry Run

Push sequence:

```
push(5)
push(3)
push(2)
```

Step 1: push(5)

- Stack empty
- push 5
- mini = 5

Stack: [5]
mini = 5

Step 2: push(3)

`3 <= mini(5)`
`encoded = 2*3 - 5 = 1`

- push **1**
- mini = 3

Stack: [5, 1]
mini = 3

👉 1 actual value nahi hai, sirf **marker** hai

Step 3: push(2)

`2 <= mini(3)`
`encoded = 2*2 - 3 = 1`

- push **1**
- mini = 2

Stack: [5, 1, 1]
mini = 2

🔥 Ab POP

Pop top = 1

- `1 < mini(2)` ⇒ encoded value hai

👉 Old mini recover karo:

```
oldMini = 2*mini - encoded
        = 2*2 - 1
        = 3
```

- mini = 3

Stack: `[5, 1]`

Pop next = 1

- `1 < mini(3)` ⇒ encoded

```
oldMini = 2*3 - 1 = 5
```

- mini = 5

Stack: `[5]`

Pop 5

- Normal pop
- Stack empty

Iska matlab kya hua?

Push time:

```
encoded = 2*newMin - oldMin
```

Pop time:

```
oldMin = 2*currentMin - encoded
```

👉 Ye **reversible encoding** hai
👉 Isliye ek hi stack kaam kar leta hai

> *We encode values smaller than the current minimum so that we can recover the previous minimum during pop.*

9. Infix to Postfix

You are given an infix expression.
Your task is to convert this infix expression into a postfix expression.

In an **infix expression**, the operator comes between operands, for example `a + b`.
In a **postfix expression**, the operator comes after operands, for example `ab+`.

Postfix expressions are easier for computers to evaluate because they do not need parentheses or precedence rules during evaluation.

Question Explanation

Given an infix expression containing operands (letters or digits), operators (`+ - * / ^`), and parentheses, convert it into a postfix expression by following operator precedence and associativity rules.

Operator precedence (high to low):

- `^`
- `* /`

- `+ -`

Associativity:

- `^` is **right associative**
- `* / + -` are **left associative**

Example Explanation

Input:
`(p + q) * (m - n)`

Step-by-step:

- Convert `(p + q)` → `pq+`
- Convert `(m - n)` → `mn-`
- Apply `*` → `pq+mn-*`

Output:
`pq+mn-*`

Approach 1

Algorithm

1. Create an empty stack to store operators.
2. Traverse the infix expression from left to right.
3. If the current character is an operand, add it to the result.
4. If the character is `'('`, push it onto the stack.

5. If the character is `')'`, pop from the stack and add to result until `'('` is found. Remove `'('`.

6. If the character is an operator:

    - Pop operators from the stack while:

        - the stack is not empty, and

        - the precedence of the current operator is **less than** the precedence of the stack top, or

        - the precedence is **equal and the operator is left associative**

    - Push the current operator onto the stack.

7. After processing the entire expression, pop all remaining operators from the stack and add them to the result.

8. The final result is the postfix expression.

Code

```
#include <bits/stdc++.h>
using namespace std;

// Function to return precedence of operators
int prec(char c) {
    if (c == '^') return 3;
    if (c == '*' || c == '/') return 2;
    if (c == '+' || c == '-') return 1;
    return -1;
}

// Function to check right associativity
bool isRightAssociative(char c) {
    return c == '^';
}
```

```
// Function to convert infix to postfix
string infixToPostfix(string s) {
    stack<char> st;
    string result = "";

    for (int i = 0; i < s.length(); i++) {
        char c = s[i];

        // Operand
        if ((c >= 'a' && c <= 'z') ||
            (c >= 'A' && c <= 'Z') ||
            (c >= '0' && c <= '9')) {
            result += c;
        }
        // Opening bracket
        else if (c == '(') {
            st.push(c);
        }
        // Closing bracket
        else if (c == ')') {
            while (!st.empty() && st.top() != '(') {
                result += st.top();
                st.pop();
            }
            st.pop(); // remove '('
        }
        // Operator
        else {
            while (!st.empty() &&
                  (prec(c) < prec(st.top()) ||
                  (prec(c) == prec(st.top()) &&
!isRightAssociative(c)))) {
                result += st.top();
                st.pop();
            }
            st.push(c);
        }
    }
```

```
    // Pop remaining operators
    while (!st.empty()) {
        result += st.top();
        st.pop();
    }

    return result;
}

int main() {
    string exp = "(p+q)*(m-n)";
    cout << infixToPostfix(exp);
    return 0;
}
```

Complexity Analysis

Time Complexity:
O(N), where N is the length of the infix expression. Each character is processed once.

Space Complexity:
O(N), due to the stack used to store operators and parentheses.

🔹 Full dry run

Input:

`(p+q)*(m-n)`

Step-by-step:

Char	Stack	Result
(	(	

p	(	p
+	(+	p
q	(+	pq
)		pq+
*	*	pq+
(	* (	pq+
m	* (	pq+m
-	* (-	pq+m
n	* (-	pq+mn
)	*	pq+mn-
end		pq+mn-*

✅ Output:

`pq+mn-*`

🎯 One-line intuition (interview ready)

> *Operators are popped from the stack if they have higher precedence, or equal precedence with left associativity.*

- Operand → output
- (→ push
-) → pop till (
- Operator:
    - pop **stronger** operators

- pop **equal & left associative**
- then push current

🔴 CORE RULE

> **Postfix me operator tab likhte hain jab usse zyada important kaam ho chuka ho.**

Stack bas **operators ko rok ke rakhne** ke liye hai.

Step 0: Sirf 3 cheezein hoti hain

Expression me aayega:

1. **Operand** (a, b, 1, 2…)
2. **Operator** (+ - * / ^)
3. **Bracket** ( )

🟢 Rule 1: Operand aaye

`a, b, x, 5`

👉 Seedha output me daal do

❓ Kyun?
Operand ko kisi ka wait nahi hota.

🟢 Rule 2: '(' aaye

👉 Stack me daal do

Bracket ka matlab:

“Iske andar ka kaam pehle complete karo”

🟢 Rule 3: ')' aaye

👉 Stack se **pop karte jao**
👉 Jab tak ' (' na mil jaye
👉 ' (' ko output me nahi daalte

🔴 Rule 4 : Operator aaye

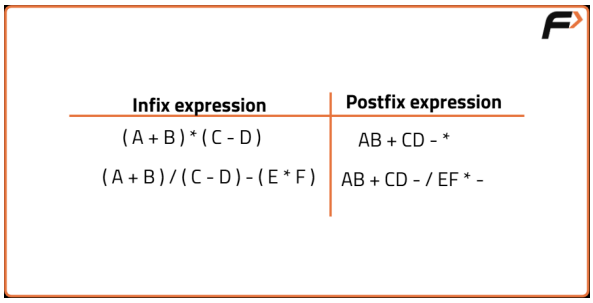
Ab dhyaan se 👇

Jab operator c aaye:

- Stack ke top ko dekho
- Agar stack top:
    - **zyada powerful** hai
    - ya **same power ka hai aur left associative** hai
      👉 to pehle usko output me daal do

Fir:
👉 current operator ko stack me push karo

Dry RUN 2



Infix expression	Postfix expression
(A+B)*(C-D)	AB+CD-*
(A+B)/(C-D)-(E*F)	AB+CD-/EF*-

2ND WALA

Symbol	Action	Stack	Output
(	push	(	
A	operand → output	(	A
+	push (top is ( )	(+	A
B	operand	(+	AB
)	pop till (		AB+
/	push	/	AB+
(	push	/ (	AB+
C	operand	/ (	AB+C
-	push	/ (-	AB+C
D	operand	/ (-	AB+CD
)	pop till (	/	AB+CD-
-	compare with / → / has higher precedence → pop		AB+CD-/
	now push -	-	AB+CD-/
(	push	- (	AB+CD-/
E	operand	- (	AB+CD-/E
*	push	- (*	AB+CD-/E
F	operand	- (*	AB+CD-/EF
)	pop till (	-	AB+CD-/EF*

end	pop remaining operators	AB+CD-/EF*-

10. Prefix to Infix Conversion

You are given a valid arithmetic expression written in **prefix notation**.
Your task is to convert it into a **fully parenthesized infix expression**.

In prefix notation, the operator comes **before** its operands.
In infix notation, the operator comes **between** operands and parentheses are used to preserve order.

Question Explanation

In prefix expressions:

- Operators appear before operands.
- Example: `+ab` means `a + b`.

To convert prefix to infix:

- We must rebuild the expression so that every operation is clearly grouped using parentheses.
- The final infix expression must be **fully parenthesized** to avoid ambiguity.

Example Explanation

Input:
`*+ab-cd`

Step-by-step:

- `+ab` becomes `(a+b)`
- `-cd` becomes `(c-d)`
- `*` combines both → `((a+b)*(c-d))`

Output:
`((a+b)*(c-d))`

Approach 1

Algorithm

1. Traverse the prefix expression from **right to left**.
2. Use a stack to store operands or partial infix expressions.
3. If the current character is an operand:
    - Push it as a string onto the stack.
4. If the current character is an operator:
    - Pop the top two elements from the stack.
    - Combine them into a new string:
      `(operand1 operator operand2)`

- Push this new string back onto the stack.

5. After the traversal ends, the stack will contain only one element.
6. That element is the required fully parenthesized infix expression.

Code

```
#include <bits/stdc++.h>
using namespace std;

// Function to convert prefix expression to infix
string prefixToInfix(string prefix) {
    stack<string> st;
    int n = prefix.size();

    // Traverse from right to left
    for(int i = n - 1; i >= 0; i--) {
        char c = prefix[i];

        // If operand, push to stack
        if(isalnum(c)) {
            st.push(string(1, c));
//string(count, character), as direct char nhi de sakte string ko
//Matlab: count baar character ko repeat karke string banao
        }
//isalnum() is a programming function (in Python, C, C++) that checks if all characters in a string or a given
//character are alphanumeric (letters 'a'-'z', 'A'-'Z', or numbers '0'-'9')
        // If operator
        else {
            string op1 = st.top();
            st.pop();
            string op2 = st.top();
```

```
            st.pop();

            string expr = "(" + op1 + c + op2 + ")";
            st.push(expr);
        }
    }

    return st.top();
}

int main() {
    string expression = "*+ab-cd";
    cout << prefixToInfix(expression);
    return 0;
}
```

Complexity Analysis

Time Complexity:
O(N), where N is the length of the prefix expression. Each character is processed once.

Space Complexity:
O(N), due to the stack storing intermediate expressions.

11. Prefix to Postfix Conversion

You are given a valid arithmetic expression in **prefix notation**.
Your task is to convert it into an equivalent **postfix expression**.

- **Prefix (Polish) notation**: Operator comes **before** operands
  Example: `+ab`

- **Postfix (Reverse Polish) notation**: Operator comes **after** operands
  Example: `ab+`

🔍 Key Idea

Prefix expressions are best handled by scanning **from right to left**.
A **stack** helps us temporarily store operands and intermediate postfix expressions.

🧠 Algorithm

1. Traverse the prefix expression **from right to left**.
2. Use a stack of strings.
3. For each character:
    - If it is an **operand**, push it onto the stack.
    - If it is an **operator**:
        - Pop two operands from the stack.
        - Combine them as:
          `operand1 + operand2 + operator`

- Push the combined string back onto the stack.

4. After traversal, the stack's top element is the final postfix expression.

✨ Example Walkthrough

Example 1

Input: +ab
Steps:

- Push b
- Push a
- Pop a, b, combine → ab+

Output: ab+

Example 2

Input: *+ab-cd

Breakdown:

- +ab → ab+
- -cd → cd-
- Combine with * → ab+cd-*

Output: `ab+cd-*`

💻 C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

// Function to convert prefix expression to postfix
string prefixToPostfix(string prefix) {
    stack<string> st;
    int n = prefix.size();

    // Traverse from right to left
    for (int i = n - 1; i >= 0; i--) {
        char c = prefix[i];

        // If operand, push to stack
        if (isalnum(c)) {
            st.push(string(1, c));
        }
        // If operator
        else {
            string op1 = st.top(); st.pop();
            string op2 = st.top(); st.pop();

            // Postfix = operand1 + operand2 + operator
            st.push(op1 + op2 + c);
        }
    }

    return st.top();
}
```

```
int main() {
    string prefix = "*+ab-cd";
    cout << "Postfix Expression: " <<
prefixToPostfix(prefix) << endl;
    return 0;
}
```

⏱️ Complexity Analysis

- **Time Complexity:** `O(n)`
  (Each character is processed once)

- **Space Complexity:** `O(n)`
  (Stack stores intermediate expressions)

12. Postfix to Prefix Conversion

You are given a **valid postfix expression**.
In postfix notation, operators come **after** operands (example: `ab+`).
Your task is to convert this postfix expression into a **prefix expression**, where operators come **before** operands (example: `+ab`).
The expression has:

- Single character operands (letters or digits)

- Binary operators (`+ - * /`)

- No spaces

- Guaranteed to be valid

Example Explanation

Example:
Postfix expression = ab+

- a → operand → push to stack
- b → operand → push to stack
- + → operator
    - pop b (op2)
    - pop a (op1)
    - form prefix: +ab
    - push +ab back to stack

Final stack top = +ab (answer)

Approach 1

Algorithm

- Create a stack of strings.
- Traverse the postfix expression **from left to right**.
- If the character is an operand, push it as a string into the stack.

- If the character is an operator:
    - Pop two elements from the stack.
    - First popped → operand2
    - Second popped → operand1
    - Form prefix expression: `operator + operand1 + operand2`
    - Push this string back into the stack.
- After traversal, the stack's top element is the prefix expression.

Code

```
#include <bits/stdc++.h>
using namespace std;

string postfixToPrefix(string postfix) {
    stack<string> st;
    for(char c : postfix) {
        if(isalnum(c)) {
            st.push(string(1,c));
        } else {
            string op2 = st.top(); st.pop();
            string op1 = st.top(); st.pop();
            st.push(c + op1 + op2);
        }
    }
    return st.top();
}

int main() {
```

```
    string postfix = "abc*+d-";
    cout << postfixToPrefix(postfix);
    return 0;
}
```

Complexity Analysis

- Time Complexity: **O(n)**
  Each character is processed once.

- Space Complexity: **O(n)**
  Stack stores intermediate expressions.

13. Postfix to Infix

You are given a **postfix expression** as a string.
In postfix notation, operators come **after** their operands (example: ab+).
Your task is to convert this postfix expression into an **equivalent infix expression** with **proper parentheses**, so that the evaluation order is preserved correctly.

The postfix expression:

- Is valid
- Contains single-character operands (letters or digits)
- Contains binary operators (+ - * /)
- Has no spaces

Example Explanation

Example 1:
Postfix = ab+c*

Step-by-step:

- a → operand → push
- b → operand → push
- + → operator
    - pop b, pop a
    - form (a+b) → push
- c → operand → push
- * → operator
    - pop c, pop (a+b)
    - form ((a+b)*c)

Final result: (a+b)*c

Approach 1

Algorithm

- Traverse the postfix expression from **left to right**.

- Use a stack of strings.
- If the character is an operand, push it as a string.
- If the character is an operator:
    - Pop the top two elements from the stack.
    - First popped → operand2
    - Second popped → operand1
    - Create a new string: `(operand1 operator operand2)`
    - Push this string back into the stack.
- After traversal, the stack's top element is the infix expression.

Code

```
#include <bits/stdc++.h>
using namespace std;

string postfixToInfix(string postfix) {
    stack<string> st;
    for(char c : postfix) {
        if(isalnum(c)) {
            st.push(string(1,c));
        } else {
            string op2 = st.top(); st.pop();
            string op1 = st.top(); st.pop();
            st.push("(" + op1 + c + op2 + ")");
        }
    }
    return st.top();
```

```
}

int main() {
    string postfix = "ab+c*";
    cout << postfixToInfix(postfix);
    return 0;
}
```

Complexity Analysis

- Time Complexity: **O(n)**
  Each character of the postfix expression is processed once.

- Space Complexity: **O(n)**
  Stack is used to store intermediate expressions.

14. Infix to Prefix

You are given an **infix expression**, where operators appear between operands (for example a+b).
Your task is to convert this infix expression into an equivalent **prefix expression**, where operators appear **before** their operands.

Prefix expressions remove the need for parentheses and make evaluation easier for computers.

Example Explanation

Example 1:
Input: `x + y * z / w + u`

- Infix has operator precedence (* and / before +)
- After applying infix to prefix conversion rules, the correct prefix form is: `++x/*yzwu`

Example 2:
Input: `a + b`
Output: `+ab`

Approach 1

Algorithm

- Reverse the infix expression.
- Replace (with) and) with (.
- Convert the modified infix expression to **postfix** using operator precedence rules.
- Reverse the postfix expression to get the **prefix** expression.
- Use a stack to handle operators during conversion.

Code

```
#include <bits/stdc++.h>
using namespace std;

bool isOperator(char c) {
    return (!isalpha(c) && !isdigit(c));
}
```

```
int getPriority(char c) {
    if(c=='+'||c=='-') return 1;
    if(c=='*'||c=='/') return 2;
    if(c=='^') return 3;
    return 0;
}

string infixToPostfix(string infix) {
    infix = "(" + infix + ")";
    stack<char> st;
    string output;

    for(char c : infix) {
        if(isalnum(c)) {
            output += c;
        } else if(c=='(') {
            st.push(c);
        } else if(c==')') {
            while(st.top()!='(') {
                output += st.top();
                st.pop();
            }
            st.pop();
        } else {
            while(getPriority(c) < getPriority(st.top())) {
                output += st.top();
                st.pop();
            }
            st.push(c);
        }
    }
    return output;
```

```
}

string infixToPrefix(string infix) {
    reverse(infix.begin(), infix.end());
    for(char &c : infix) {
        if(c=='(') c=')';
        else if(c==')') c='(';
    }
    string prefix = infixToPostfix(infix);
    reverse(prefix.begin(), prefix.end());
    return prefix;
}

int main() {
    string s = "x+y*z/w+u";
    cout << infixToPrefix(s);
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** O(N)
  Each character is processed a constant number of times.

- **Space Complexity:** O(N)
  Stack space used for operators during conversion.

15. Next Greater Element Using Stack

You are given an integer array.
For every element in the array, you have to find the **next greater element on its right side**.
The next greater element of a number x is the first number greater than x that appears **after it** while moving from left to right.
If no such element exists, return `-1` for that position.

Example Explanation

Example 1:
Input: `[1, 3, 2, 4]`

- For 1, the next greater element is 3
- For 3, the next greater element is 4
- For 2, the next greater element is 4
- For 4, no greater element exists → `-1`

Output: `[3, 4, 4, -1]`

Example 2:
Input: `[6, 8, 0, 1, 3]`

- `6 → 8`
- `8 → -1`
- `0 → 1`

- 1 → 3
- 3 → -1

Output: `[8, -1, 1, 3, -1]`

Approach 1

Algorithm

- Use a stack to keep elements in decreasing order.
- Traverse the array from **right to left**.
- For each element:
    - Remove all elements from the stack that are **smaller than or equal** to the current element.
    - If the stack becomes empty, no greater element exists → store `-1`.
    - Otherwise, the top of the stack is the next greater element.
- Push the current element into the stack.
- Continue until all elements are processed.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
```

```
public:
    vector<int> nextGreater(vector<int>& nums) {
        stack<int> st;
        int n = nums.size();
        vector<int> res(n);

        for(int i = n - 1; i >= 0; i--) {
            while(!st.empty() && st.top() <= nums[i]) {
                st.pop();
            }

            if(st.empty()) res[i] = -1;
            else res[i] = st.top();

            st.push(nums[i]);
        }
        return res;
    }
};

int main() {
    vector<int> nums = {1, 3, 2, 4};
    Solution sol;
    vector<int> ans = sol.nextGreater(nums);

    for(int x : ans) {
        cout << x << " ";
    }
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** O(N)
  Each element is pushed and popped from the stack at most once.

- **Space Complexity:** O(N)
  Stack and result array both use linear space.

---

16. Next Greater Element – 2

You are given a **circular integer array**.
For every element, you have to find the **next greater element** while moving in a **clockwise (circular) manner**.

The next greater element of a number x is the **first number greater than x** that appears when we move forward in the array.
If we reach the end, we continue from the beginning because the array is circular.
If no such element exists, return `-1` for that position.

Example Explanation

Example 1:
Input: `[5, 7, 1, 7, 6, 0]`

- For 5 → next greater is 7

- For 7 → no greater element exists → -1

- For 1 → next greater is 7
- For 7 → no greater element exists → `-1`
- For 6 → next greater is 7
- For `0` → next greater is 5 (found after circular traversal)

Output: `[7, -1, 7, -1, 7, 5]`

Approach 1 (Brute Force)

Algorithm

- Create an answer array of size `n`, initialized with `-1`.
- For each element at index `i`:
    - Traverse the next `n-1` elements in circular order using `(i + j) % n`.
    - If a greater element is found, store it in `ans[i]` and stop searching.
- After checking all elements, return the answer array.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
```

```
    vector<int> nextGreaterElements(vector<int> arr) {
        int n = arr.size();
        vector<int> ans(n, -1);

        for(int i = 0; i < n; i++) {
            for(int j = 1; j < n; j++) {
                int ind = (i + j) % n;
                if(arr[ind] > arr[i]) {
                    ans[i] = arr[ind];
                    break;
                }
            }
        }
        return ans;
    }
};

int main() {
    vector<int> arr = {5, 7, 1, 7, 6, 0};
    Solution sol;
    vector<int> ans = sol.nextGreaterElements(arr);

    for(int x : ans) cout << x << " ";
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** $O(N^2)$
  For each element, we may scan the entire array.

- **Space Complexity:** O(N)
  Used for the result array.

Approach 2 (Optimal Using Stack)

Algorithm

- Initialize an answer array with `-1`.
- Use a stack to maintain elements in decreasing order.
    - Traverse the array from index `2*n - 1` to `0` to simulate circular traversal.
- Use `i % n` to get the actual index.
- While stack is not empty and top ≤ current element, pop from stack.
- If `i < n`:
    - If stack is empty → answer is `-1`
    - Else → top of stack is the next greater element
- Push current element into the stack.
- Return the answer array.

Code

```
#include <bits/stdc++.h>
using namespace std;
```

```cpp
class Solution {
public:
    vector<int> nextGreaterElements(vector<int> arr) {
        int n = arr.size();
        vector<int> ans(n, -1);
        stack<int> st;

        for(int i = 2*n - 1; i >= 0; i--) {
            int ind = i % n;

            while(!st.empty() && st.top() <= arr[ind]) {
                st.pop();
            }

            if(i < n) {
                if(!st.empty()) ans[ind] = st.top();
            }

            st.push(arr[ind]);
        }
        return ans;
    }
};

int main() {
    vector<int> arr = {5, 7, 1, 7, 6, 0};
    Solution sol;
    vector<int> ans = sol.nextGreaterElements(arr);

    for(int x : ans) cout << x << " ";
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** O(N)
  Each element is pushed and popped from the stack at most once.

- **Space Complexity:** O(N)
  Stack and answer array both take linear space.

Dry Run

arr = [1, 2, 1]
n = 3
Loop: i = 5 → 0

i	idx	arr[idx]	Stack before	Action	Stack after	ans
5	2	1	[]	push	[1]	[-1,-1,-1]
4	1	2	[1]	pop 1, push 2	[2]	[-1,-1,-1]
3	0	1	[2]	push	[2,1]	[-1,-1,-1]
2	2	1	[2,1]	pop 1	[2]	ans[2]=2
1	1	2	[2]	pop 2	[]	ans[1]=-1
0	0	1	[]	ans[0]=2		

17. Next Smaller Element

You are given an integer array `arr`.
For every element in the array, you have to find its **Next Smaller Element (NSE)**.

The **Next Smaller Element** of an element x is the **first element to the right of x that is strictly smaller than x**.
If no such element exists on the right side, the answer for that element is `-1`.

Example Explanation

Example 1:
Input: `[4, 8, 5, 2, 25]`

- For 4 → next smaller is 2
- For 8 → next smaller is 5
- For 5 → next smaller is 2
- For 2 → no smaller element on the right → `-1`
- For 25 → no smaller element → `-1`

Output: `[2, 5, 2, -1, -1]`

Approach 1 (Brute Force)

Algorithm

- Create an answer array of size n and initialize all values with `-1`
- For each index `i` from `0` to `n-1`
    - Traverse elements from `i+1` to `n-1`

    - If an element smaller than `arr[i]` is found
        - Store it in `ans[i]`
        - Stop searching further
- Return the answer array

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    vector<int> nextSmallerElement(vector<int>& arr) {
        int n = arr.size();
        vector<int> ans(n, -1);

        for(int i = 0; i < n; i++) {
            for(int j = i + 1; j < n; j++) {
                if(arr[j] < arr[i]) {
                    ans[i] = arr[j];
                    break;
                }
            }
        }
        return ans;
    }
};

int main() {
    vector<int> arr = {4, 8, 5, 2, 25};
```

```
    Solution sol;
    vector<int> ans = sol.nextSmallerElement(arr);

    for(int x : ans) cout << x << " ";
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** O(N²)
  For each element, we may scan the remaining array.

- **Space Complexity:** O(N)
  Used for the result array.

Approach 2 (Optimal Using Stack)

Algorithm

- Create an answer array initialized with −1
- Use a stack to keep elements in increasing order
- Traverse the array from right to left
- While stack is not empty and stack top ≥ current element, pop the stack
- If stack is not empty, top of stack is the next smaller element
- Push current element into the stack
- Return the answer array

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    vector<int> nextSmallerElement(vector<int>& arr) {
        int n = arr.size();
        vector<int> ans(n, -1);
        stack<int> st;

        for(int i = n - 1; i >= 0; i--) {
            while(!st.empty() && st.top() >= arr[i]) {
                st.pop();
            }
            if(!st.empty()) {
                ans[i] = st.top();
            }
            st.push(arr[i]);
        }
        return ans;
    }
};

int main() {
    vector<int> arr = {4, 8, 5, 2, 25};
    Solution sol;
    vector<int> ans = sol.nextSmallerElement(arr);

    for(int x : ans) cout << x << " ";
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** O(N)
  Each element is pushed and popped at most once.

- **Space Complexity:** O(N)
  Stack and answer array require linear space.

18. Number of NGEs to the Right

You are given an integer array `arr` of size `n`.
For every element in the array, you have to find its **Next Greater Element (NGE)** on the right side.

The **Next Greater Element** of an element x is the **nearest element to the right of x that is strictly greater than x**.
If no such element exists, return `-1` for that position.

Example Explanation

Example 1:
Input: `[1, 3, 2, 4]`

- For 1, the next greater element is 3

- For 3, the next greater element is 4

- For 2, the next greater element is 4

- For 4, there is no greater element on the right → `-1`

Output: `[3, 4, 4, -1]`

Approach 1 (Brute Force)

Algorithm

- Create an answer array initialized with `-1`
- Traverse the array from left to right
- For each element at index `i`, check elements to its right
- The first element greater than the current one is the NGE
- If found, store it and stop checking further
- Return the answer array

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    vector<int> nextLargerElement(vector<int> arr) {
        int n = arr.size();
        vector<int> ans(n, -1);

        for(int i = 0; i < n; i++) {
            for(int j = i + 1; j < n; j++) {
```

```
                if(arr[j] > arr[i]) {
                    ans[i] = arr[j];
                    break;
                }
            }
        }
        return ans;
    }
};

int main() {
    vector<int> arr = {1, 3, 2, 4};
    Solution sol;
    vector<int> ans = sol.nextLargerElement(arr);

    for(int x : ans) cout << x << " ";
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** $O(N^2)$
  For each element, we may scan all elements to its right.

- **Space Complexity:** $O(N)$
  Used for the answer array.

Approach 2 (Optimal Using Stack)

Algorithm

- Create an answer array
- Use a stack to maintain elements in decreasing order
- Traverse the array from right to left
- While stack is not empty and top ≤ current element, pop the stack
- If stack becomes empty, NGE is −1
- Otherwise, top of stack is the NGE
- Push current element into the stack
- Return the answer array

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    vector<int> nextLargerElement(vector<int> arr) {
        int n = arr.size();
        vector<int> ans(n);
        stack<int> st;

        for(int i = n - 1; i >= 0; i--) {
            while(!st.empty() && st.top() <= arr[i]) {
                st.pop();
            }
            if(st.empty()) ans[i] = -1;
            else ans[i] = st.top();
            st.push(arr[i]);
```

```
        }
        return ans;
    }
};

int main() {
    vector<int> arr = {1, 3, 2, 4};
    Solution sol;
    vector<int> ans = sol.nextLargerElement(arr);

    for(int x : ans) cout << x << " ";
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** O(N)
  Each element is pushed and popped at most once.

- **Space Complexity:** O(N)
  Stack and answer array use linear space.

19. Number of NGEs to the Right

You are given an array of `N` integers and `Q` queries.
Each query gives an index, and for that index you have to find **how many elements to the right are strictly greater than the element at that index**.

In simple words, for a given index `i`, count all `j` such that:

- `j > i`
- `arr[j] > arr[i]`

Example Explanation

Input array:
`arr = {3, 4, 2, 7, 5, 8, 10, 6}`

Queries:

- index = `0`
- index = 5

For index `0` (value = 3):
Elements to the right that are greater than 3 → `4, 7, 5, 8, 10, 6`
Count = 6

For index 5 (value = 8):
Elements to the right that are greater than 8 → `10`
Count = 1

Output:
`6 1`

Approach 1 (Naive Approach)

Algorithm

- For every query index:

- Traverse the array from `index + 1` to the end
- Count how many elements are greater than `arr[index]`

- Print the count for each query

Code

```
#include <bits/stdc++.h>
using namespace std;

int nextGreaterElements(vector<int>& a, int index) {
    int count = 0;
    int n = a.size();
    for(int i = index + 1; i < n; i++) {
        if(a[i] > a[index]) {
            count++;
        }
    }
    return count;
}

int main() {
    vector<int> a = {3, 4, 2, 7, 5, 8, 10, 6};
    vector<int> queries = {0, 5};

    for(int idx : queries) {
        cout << nextGreaterElements(a, idx) << " ";
    }
    return 0;
}
```

- **Time Complexity:** `O(N * Q)`
  For each query, we may traverse the entire remaining array.

- **Space Complexity:** `O(1)`
  No extra space used apart from variables.

Approach 2 (Optimized Using Merge Sort Concept)

Algorithm

- Create a vector of pairs `(value, original_index)`
- Use a modified merge sort:
    - While merging, if `left_value < right_value`
        - Then all remaining elements in the right half are greater
        - Add that count to `ans[original_index]`
- This gives the count of elements greater on the right for **every index**
- For each query, simply print the stored answer

Code

```
#include <bits/stdc++.h>
using namespace std;

void merge(vector<pair<int,int>>& vec, vector<int>& ans,
           int low, int mid, int high) {

    vector<pair<int,int>> left, right;
```

```
    for(int i = low; i <= mid; i++) left.push_back(vec[i]);
    for(int i = mid + 1; i <= high; i++)
right.push_back(vec[i]);

    int i = 0, j = 0, k = low;

    while(i < left.size() && j < right.size()) {
        if(left[i].first < right[j].first) {
            ans[left[i].second] += (right.size() - j);
            vec[k++] = left[i++];
        } else {
            vec[k++] = right[j++];
        }
    }

    while(i < left.size()) vec[k++] = left[i++];
    while(j < right.size()) vec[k++] = right[j++];
}

void mergeSort(vector<pair<int,int>>& vec, vector<int>& ans,
               int low, int high) {
    if(low >= high) return;

    int mid = (low + high) / 2;
    mergeSort(vec, ans, low, mid);
    mergeSort(vec, ans, mid + 1, high);
    merge(vec, ans, low, mid, high);
}

void nextGreaterElements(int n, vector<int>& nums,
                         vector<int>& queries) {
```

```
    vector<pair<int,int>> vec;
    for(int i = 0; i < n; i++) {
        vec.push_back({nums[i], i});
    }

    vector<int> ans(n, 0);
    mergeSort(vec, ans, 0, n - 1);

    for(int idx : queries) {
        cout << ans[idx] << " ";
    }
}

int main() {
    vector<int> nums = {3, 4, 2, 7, 5, 8, 10, 6};
    vector<int> queries = {0, 5};

    nextGreaterElements(nums.size(), nums, queries);
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** `O(N log N)`
  Due to merge sort.

- **Space Complexity:** `O(N)`
  Extra arrays used during merge process.

20. Trapping Rainwater

You are given an array `height` where each element represents the height of a bar.
All bars have width 1. After raining, water can be trapped between these bars.

Your task is to calculate **the total units of water that can be trapped**.

Water can only be trapped if there are **taller bars on both the left and right sides** of a position.

Example Explanation

Example 1

Input:
`height = [0,1,0,2,1,0,1,3,2,1,2,1]`

At each index, water trapped depends on the minimum of the tallest bar on the left and right.

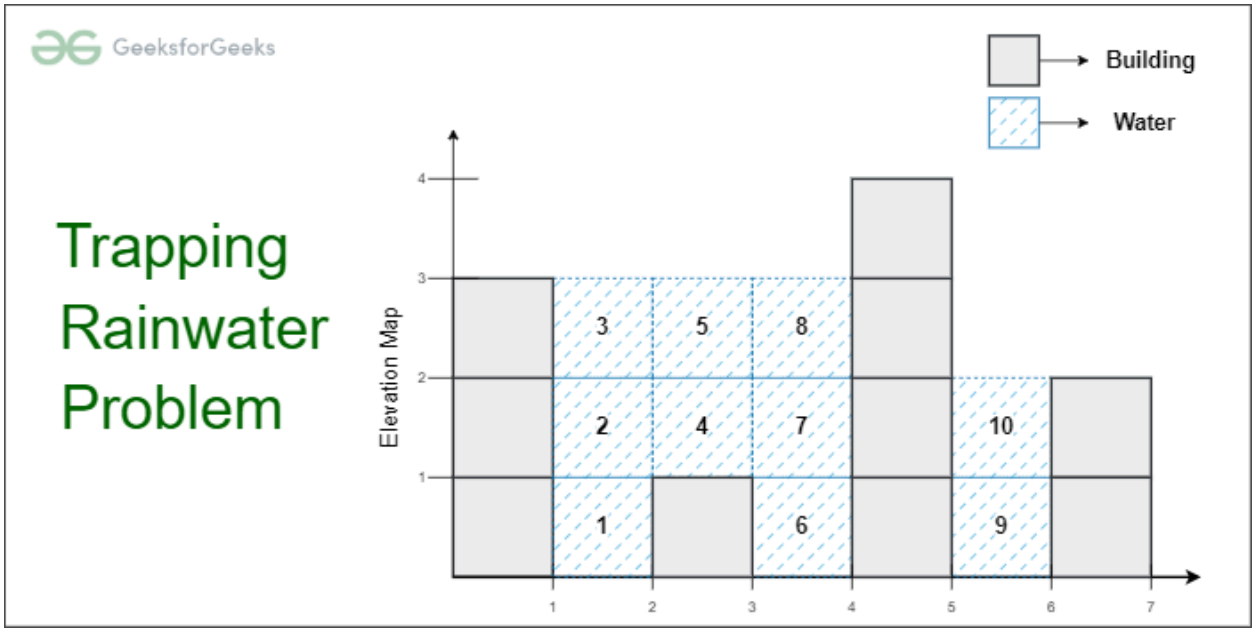
Total trapped water = 6 units.

Example 2

Input:
`height = [4,2,0,3,2,5]`

Water gets trapped between taller bars on both sides.

Total trapped water = 9 units.



Approach 1 (Brute Force)

Algorithm

- For every index `i`:
    - Find the maximum height on the left of `i`
    - Find the maximum height on the right of `i`
    - Water at `i` = `min(maxLeft, maxRight) - height[i]`
- Add water for all indices

Code

```
#include <bits/stdc++.h>
using namespace std;
```

```
class Solution {
public:
    int trap(vector<int>& height) {
        int n = height.size();
        int totalWater = 0;

        for(int i = 0; i < n; i++) {
            int maxLeft = 0, maxRight = 0;

            for(int j = 0; j <= i; j++)
                maxLeft = max(maxLeft, height[j]);

            for(int j = i; j < n; j++)
                maxRight = max(maxRight, height[j]);

            totalWater += min(maxLeft, maxRight) -
height[i];
        }
        return totalWater;
    }
};

int main() {
    vector<int> height = {0,1,0,2,1,0,1,3,2,1,2,1};
    Solution sol;
    cout << sol.trap(height);
    return 0;
}
```

- **Time Complexity:** $O(N^2)$
  For each bar, we scan left and right.

- **Space Complexity:** $O(1)$

Ek approach to use prefix and suffix max array, isme prefix array ki jarurat hai nhi wo to traverse karte hue hi kaam chal jayega

---

Approach 2 (Optimal Two Pointer Approach)

Intuition: we only need one (either left or right) to calculate water at given index

Algorithm

- Initialize two pointers: `left = 0`, `right = n - 1`
- Maintain `maxLeft` and `maxRight`
- While `left <= right`:
    - If `height[left] <= height[right]`:
        - If `height[left] >= maxLeft`, update `maxLeft`
        - Else add `maxLeft - height[left]` to answer
        - Move `left`
    - Else:
        - If `height[right] >= maxRight`, update `maxRight`

- Else add `maxRight - height[right]` to answer
- Move `right`

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int trap(vector<int>& height) {
        int n = height.size();
        int left = 0, right = n - 1;
        int maxLeft = 0, maxRight = 0;
        int totalWater = 0;

        while(left <= right) {
            if(height[left] <= height[right]) {
                if(height[left] >= maxLeft)
                    maxLeft = height[left];
                else
                    totalWater += maxLeft - height[left];
                left++;
            } else {
                if(height[right] >= maxRight)
                    maxRight = height[right];
                else
                    totalWater += maxRight - height[right];
                right--;
            }
        }
```

```
        return totalWater;
    }
};

int main() {
    vector<int> height = {0,1,0,2,1,0,1,3,2,1,2,1};
    Solution sol;
    cout << sol.trap(height);
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** $O(N)$
  Each element is processed once.

- **Space Complexity:** $O(1)$

21. Sum of Subarray Minimums

You are given an integer array `arr` of size n.
For **every possible contiguous subarray**, find the **minimum element** of that subarray and **add all those minimum values together**.

Since the answer can be very large, return it modulo $10^9 + 7$.

In simple words:
👉 Generate all subarrays → find the minimum in each → sum them all.

Example Explanation

Example 1

Input:
`arr = [3, 1, 2, 5]`

All subarrays and their minimums:

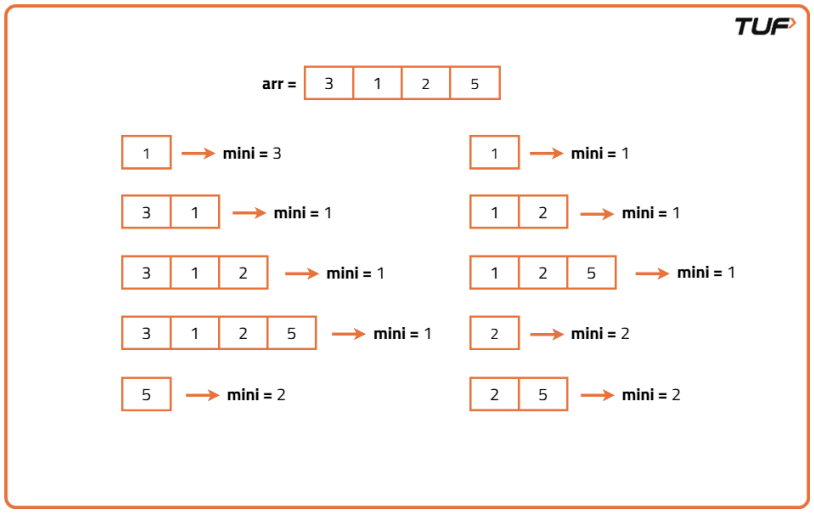
- `[3]` → 3
- `[1]` → 1
- `[2]` → 2
- `[5]` → 5
- `[3,1]` → 1
- `[1,2]` → 1
- `[2,5]` → 2
- `[3,1,2]` → 1
- `[1,2,5]` → 1
- `[3,1,2,5]` → 1

Sum = 18

Approach 1 (Brute Force)

Algorithm

- Initialize `sum = 0`
- Fix the starting index `i`
- Initialize `mini = arr[i]`
- Extend the subarray to the right using index `j`
- Update `mini` for every extension
- Add `mini` to `sum`
- Repeat for all starting indices



Code

```
#include <bits/stdc++.h>
using namespace std;
```

```
class Solution {
public:
    int sumSubarrayMins(vector<int>& arr) {
        int n = arr.size();
        int mod = 1e9 + 7;
        int sum = 0;

        for(int i = 0; i < n; i++) {
            int mini = arr[i];
            for(int j = i; j < n; j++) {
                mini = min(mini, arr[j]);
                sum = (sum + mini) % mod;
            }
        }
        return sum;
    }
};

int main() {
    vector<int> arr = {3,1,2,5};
    Solution sol;
    cout << sol.sumSubarrayMins(arr);
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** $O(N^2)$
  Two nested loops generate all subarrays.

- **Space Complexity:** $O(1)$
  Only variables are used.

Approach 2 (Optimal Using Stack)

Algorithm

Key idea:
Instead of generating subarrays, **count how many subarrays consider `arr[i]` as the minimum**.

Steps:

- Find **Next Smaller Element (NSE)** index for every element
- Find **Previous Smaller or Equal Element (PSEE)** index for every element
- For each index `i`:
  - Left choices = `i - psee[i]`
  - Right choices = `nse[i] - i`
  - Total subarrays where `arr[i]` is minimum = `left * right`
- Contribution = `arr[i] * left * right`
- Add all contributions modulo $10^9 + 7$

Code

```
#include <bits/stdc++.h>
using namespace std;
```

```
class Solution {
private:
    vector<int> findNSE(vector<int>& arr) {
        int n = arr.size();
        vector<int> ans(n);
        stack<int> st;

        for(int i = n - 1; i >= 0; i--) {
            while(!st.empty() && arr[st.top()] >= arr[i])
                st.pop();
            ans[i] = st.empty() ? n : st.top();
            st.push(i);
        }
        return ans;
    }

    vector<int> findPSEE(vector<int>& arr) {
        int n = arr.size();
        vector<int> ans(n);
        stack<int> st;

        for(int i = 0; i < n; i++) {
            while(!st.empty() && arr[st.top()] > arr[i])
                st.pop();
            ans[i] = st.empty() ? -1 : st.top();
            st.push(i);
        }
        return ans;
    }

public:
```

```
    int sumSubarrayMins(vector<int>& arr) {
        int n = arr.size();
        int mod = 1e9 + 7;

        vector<int> nse = findNSE(arr);
        vector<int> psee = findPSEE(arr);

        long long sum = 0;

        for(int i = 0; i < n; i++) {
            long long left = i - psee[i];
            long long right = nse[i] - i;
            sum = (sum + (left * right % mod) * arr[i]) %
mod;
        }
        return sum;
    }
};

int main() {
    vector<int> arr = {3,1,2,5};
    Solution sol;
    cout << sol.sumSubarrayMins(arr);
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** $O(N)$
  Each element is pushed and popped once from stack.

- **Space Complexity:** `O(N)`
  Stack and helper arrays are used.

22. Asteroid Collision

You are given an integer array `asteroids`.
Each element represents an asteroid moving in a straight line.

- **Absolute value** → size of the asteroid
- **Sign** → direction
  - Positive (+) → moving right
  - Negative (-) → moving left

All asteroids move at the same speed.

When two asteroids collide:

- Smaller asteroid explodes
- If both have the same size, both explode
- Asteroids moving in the same direction never collide

You need to return the **final state of asteroids after all collisions**.

Example Explanation

Example 1

Input:
`[2, -2]`

- 2 moves right, `-2` moves left
- Same size → both explode

Output:
`[]`

Example 2

Input:
`[10, 20, -10]`

- `10` and `20` move right
- `-10` moves left and collides with `20`
- `20` is larger → `-10` explodes

Final result:
`[10, 20]`

Approach 1

Algorithm

- Use a stack to keep track of asteroids after collisions
- Traverse the array from left to right
- If the asteroid is moving right (`> 0`), push it into the stack
- If the asteroid is moving left (`< 0`):
    - While the stack is not empty and the top is a smaller right-moving asteroid, pop it
    - If the top asteroid is the same size, pop it and do not push current
    - If the stack is empty or the top is left-moving, push the current asteroid
- After processing all asteroids, the stack contains the final state

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    vector<int> asteroidCollision(vector<int>& asteroids) {
        vector<int> st;

        for(int i = 0; i < asteroids.size(); i++) {
            if(asteroids[i] > 0) {
                st.push_back(asteroids[i]);
            } else {
                while(!st.empty() && st.back() > 0 &&
st.back() < abs(asteroids[i])) {
                    st.pop_back();
```

```
                }

                if(!st.empty() && st.back() ==
abs(asteroids[i])) {
                    st.pop_back();
                }
                else if(st.empty() || st.back() < 0) {
                    st.push_back(asteroids[i]);
                }
            }
        }
        return st;
    }
};

int main() {
    vector<int> asteroids = {10, 20, -10};
    Solution sol;

    vector<int> ans = sol.asteroidCollision(asteroids);

    for(int x : ans) {
        cout << x << " ";
    }
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** $O(N)$
  Each asteroid is pushed and popped at most once.

- **Space Complexity:** `O(N)`
  Stack can store all asteroids in the worst case.

23. Sum of Subarray Ranges

You are given an integer array `nums`.
For every **contiguous subarray**, its **range** is defined as:

```
range = maximum element - minimum element
```

Your task is to calculate the **sum of ranges of all possible subarrays**.

A subarray must be **non-empty and contiguous**.

Example Explanation

Example 1

Input:
`nums = [1, 2, 3]`

All subarrays and their ranges:

- [1] → 0
- [2] → 0
- [3] → 0
- [1,2] → 2 − 1 = 1

- `[2,3]` → 3 − 2 = 1
- `[1,2,3]` → 3 − 1 = 2

Sum = `0 + 0 + 0 + 1 + 1 + 2 = 4`

Output:
4

Approach 1 (Brute Force)

Algorithm

- Initialize a variable `sum = 0`
- Fix the starting index `i` of the subarray
- Initialize `smallest` and `largest` as `arr[i]`
- Extend the subarray to the right using index `j`
- Update `smallest` and `largest`
- Add `(largest - smallest)` to `sum`
- Repeat for all possible subarrays
- Return `sum`

Code

```
#include <bits/stdc++.h>
using namespace std;
```

```
class Solution {
public:
    long long subArrayRanges(vector<int>& arr) {
        int n = arr.size();
        long long sum = 0;

        for(int i = 0; i < n; i++) {
            int smallest = arr[i];
            int largest = arr[i];

            for(int j = i; j < n; j++) {
                smallest = min(smallest, arr[j]);
                largest = max(largest, arr[j]);
                sum += (largest - smallest);
            }
        }
        return sum;
    }
};

int main() {
    vector<int> arr = {1, 2, 3};
    Solution sol;
    cout << sol.subArrayRanges(arr);
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** $O(N^2)$

- **Space Complexity:** $O(1)$

Approach 2 (Optimal Using Monotonic Stack)

Algorithm

- Every element contributes to the final answer as:
    - how many subarrays where it is **maximum**
    - how many subarrays where it is **minimum**
- Total answer =
  `sum of subarray maximums - sum of subarray minimums`

Steps:

1. Find **Next Smaller Element (NSE)** for each index
2. Find **Previous Smaller or Equal Element (PSEE)**
3. Find **Next Greater Element (NGE)**
4. Find **Previous Greater or Equal Element (PGEE)**
5. Use these indices to count how many subarrays each element is minimum and maximum
6. Compute total maximum contribution
7. Compute total minimum contribution
8. Subtract both results

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
private:
    vector<int> findNSE(vector<int>& arr) {
        int n = arr.size();
        vector<int> ans(n);
        stack<int> st;

        for(int i = n - 1; i >= 0; i--) {
            while(!st.empty() && arr[st.top()] >= arr[i])
st.pop();
            ans[i] = st.empty() ? n : st.top();
            st.push(i);
        }
        return ans;
    }

    vector<int> findPSEE(vector<int>& arr) {
        int n = arr.size();
        vector<int> ans(n);
        stack<int> st;

        for(int i = 0; i < n; i++) {
            while(!st.empty() && arr[st.top()] > arr[i])
st.pop();
            ans[i] = st.empty() ? -1 : st.top();
            st.push(i);
        }
        return ans;
```

```
    }

    vector<int> findNGE(vector<int>& arr) {
        int n = arr.size();
        vector<int> ans(n);
        stack<int> st;

        for(int i = n - 1; i >= 0; i--) {
            while(!st.empty() && arr[st.top()] <= arr[i])
st.pop();
            ans[i] = st.empty() ? n : st.top();
            st.push(i);
        }
        return ans;
    }

    vector<int> findPGEE(vector<int>& arr) {
        int n = arr.size();
        vector<int> ans(n);
        stack<int> st;

        for(int i = 0; i < n; i++) {
            while(!st.empty() && arr[st.top()] < arr[i])
st.pop();
            ans[i] = st.empty() ? -1 : st.top();
            st.push(i);
        }
        return ans;
    }

    long long sumSubarrayMins(vector<int>& arr) {
        vector<int> nse = findNSE(arr);
```

```
        vector<int> psee = findPSEE(arr);
        long long sum = 0;

        for(int i = 0; i < arr.size(); i++) {
            long long left = i - psee[i];
            long long right = nse[i] - i;
            sum += left * right * arr[i];
        }
        return sum;
    }

    long long sumSubarrayMaxs(vector<int>& arr) {
        vector<int> nge = findNGE(arr);
        vector<int> pgee = findPGEE(arr);
        long long sum = 0;

        for(int i = 0; i < arr.size(); i++) {
            long long left = i - pgee[i];
            long long right = nge[i] - i;
            sum += left * right * arr[i];
        }
        return sum;
    }

public:
    long long subArrayRanges(vector<int>& arr) {
        return sumSubarrayMaxs(arr) - sumSubarrayMins(arr);
    }
};

int main() {
    vector<int> arr = {1, 2, 3};
```

```
    Solution sol;
    cout << sol.subArrayRanges(arr);
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** `O(N)`
- **Space Complexity:** `O(N)`

24. **Remove K Digits**

You are given a string `nums` that represents a non-negative integer and an integer `k`.
Your task is to remove exactly `k` digits from `nums` such that the resulting number is the **smallest possible**.

The final answer:

- must not contain **leading zeroes**
- if all digits are removed or only zeroes remain, return `"0"`

Example Explanation

Example 1

Input:
`nums = "541892", k = 2`

We want the smallest number after removing 2 digits.

Steps:

- Remove 5 because next digit 4 is smaller
- Remove 4 because next digit 1 is smaller

Remaining digits → `"1892"`

Output:
`"1892"`

Example 2

Input:
`nums = "1002991", k = 3`

Steps:

- Remove leading 1
- Remove 9 and 9
- Remaining digits → `"0021"`
- Remove leading zeroes → `"21"`

Output:
`"21"`

Approach 1 - Number ko minimum banana hai, isliye left se bade digits ko hatao jab unke right me chhota digit mil jaaye.

Algorithm

- Use a stack to build the smallest number
- Traverse digits from left to right
- While:
    - stack is not empty
    - `k > 0`
    - top of stack is greater than current digit
      → pop from stack and decrement k
- Push the current digit into the stack
- If `k > 0` after traversal, remove last k digits from stack
- Build the result from the stack
- Remove leading zeroes
- If result becomes empty, return `"0"`

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
```

```
public:
    string removeKdigits(string nums, int k) {
        stack<char> st;

        for(int i = 0; i < nums.size(); i++) {
            char digit = nums[i];

            while(!st.empty() && k > 0 && st.top() > digit)
{
                st.pop();
                k--;
            }
            st.push(digit);
        }

        while(!st.empty() && k > 0) {
            st.pop();
            k--;
        }

        if(st.empty()) return "0";

        string res = "";
        while(!st.empty()) {
            res.push_back(st.top());
            st.pop();
        }

        while(res.size() > 0 && res.back() == '0') {
            res.pop_back();
        }
```

```
        reverse(res.begin(), res.end());

        if(res.empty()) return "0";
        return res;
    }
};

int main() {
    string nums = "541892";
    int k = 2;

    Solution sol;
    cout << sol.removeKdigits(nums, k);
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** `O(N)`
  Each digit is pushed and popped at most once.

- **Space Complexity:** `O(N)`
  Stack can store up to all digits in the worst case.

25. Area of Largest Rectangle in Histogram

You are given an array `heights[]` where each element represents the height of a bar in a histogram.

Each bar has a width of 1.
Your task is to find the **maximum rectangular area** that can be formed using contiguous bars in the histogram.

Example Explanation

Input:
`heights = [2, 1, 5, 6, 2, 3]`

Possible rectangles:

- Using bar 2 → area = 2 × 1 = 2
- Using bar 1 across all bars → area = 1 × 6 = 6
- Using bars `[5,6]` → min height = 5, width = 2 → area = 10
- Other combinations give smaller areas

So, the **maximum area = 10**.

Approach 1: Brute Force

Algorithm

- Consider every bar as the starting point.
- Extend to the right and keep track of the minimum height in the current range.
- For each range, calculate area = minimum height × width.
- Keep updating the maximum area.

Code

```
#include <bits/stdc++.h>
using namespace std;

int largestarea(int arr[], int n) {
    int maxArea = 0;

    for (int i = 0; i < n; i++) {
        int minHeight = INT_MAX;

        for (int j = i; j < n; j++) {
            minHeight = min(minHeight, arr[j]);
            int width = j - i + 1;
            maxArea = max(maxArea, minHeight * width);
        }
    }
    return maxArea;
}

int main() {
    int arr[] = {2, 1, 5, 6, 2, 3};
    int n = 6;
    cout << largestarea(arr, n);
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** $O(N^2)$
- **Space Complexity:** O(1)

Approach 2: Optimized Using Previous & Next Smaller Elements

Algorithm

- For each bar, find:
    - nearest smaller element to the left
    - nearest smaller element to the right
- Width = rightSmaller − leftSmaller + 1
- Area = height × width
- Take the maximum among all bars.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int largestRectangleArea(vector<int> &heights) {
        int n = heights.size();
        stack<int> st;
        vector<int> left(n), right(n);

        for (int i = 0; i < n; i++) {
            while (!st.empty() && heights[st.top()] >=
heights[i])
                st.pop();
```

```
            left[i] = st.empty() ? 0 : st.top() + 1;
            st.push(i);
        }

        while (!st.empty()) st.pop();

        for (int i = n - 1; i >= 0; i--) {
            while (!st.empty() && heights[st.top()] >=
heights[i])
                st.pop();
            right[i] = st.empty() ? n - 1 : st.top() - 1;
            st.push(i);
        }

        int maxA = 0;
        for (int i = 0; i < n; i++) {
            int width = right[i] - left[i] + 1;
            maxA = max(maxA, heights[i] * width);
        }
        return maxA;
    }
};

int main() {
    vector<int> heights = {2, 1, 5, 6, 2, 3};
    Solution obj;
    cout << obj.largestRectangleArea(heights);
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** O(N)
- **Space Complexity:** O(N)

Approach 3: Single Pass Stack Approach

Algorithm

- Use a stack to store indices of increasing bars.
- Traverse the array:
    - If current bar is smaller, pop from stack and calculate area.
    - Width is decided using current index and new stack top.
- Add an imaginary bar of height `0` at the end to flush the stack.
- Track the maximum area.

Jab tak bars increasing hain, hum wait karte hain.
Jaise hi koi chhoti bar milti hai,
toh pichhli badi bars ka “right boundary” mil jaata hai.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int largestRectangleArea(vector<int> &histo) {
```

```
        stack<int> st;
        int maxA = 0;
        int n = histo.size();

        for (int i = 0; i <= n; i++) {
            while (!st.empty() && (i == n || histo[st.top()]
>= histo[i])) {
                int height = histo[st.top()];
                st.pop();

                int width = st.empty() ? i : i - st.top()-1;
                maxA = max(maxA, height * width);
            }
            st.push(i);
        }
        return maxA;
    }
};

int main() {
    vector<int> histo = {2, 1, 5, 6, 2, 3};
    Solution obj;
    cout << obj.largestRectangleArea(histo);
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** O(N)
- **Space Complexity:** O(N)

26. Maximal Rectangles

You are given a binary matrix of size `m × n` containing only `0` and `1`.
Your task is to find the **largest rectangle** that contains **only 1's** and return its area.

A rectangle must be formed by **contiguous rows and columns**.

Question Explanation

Each cell in the matrix can either contribute to a rectangle (if it is 1) or break it (if it is `0`).
To solve this, we convert each row into a **histogram of heights** where each height represents the number of consecutive 1s above that cell (including itself).

Then, for every row's histogram, we calculate the **largest rectangle in a histogram** and keep track of the maximum area found.

Example Explanation

Input

```
matrix =
[
 [1,0,1,0,0],
 [1,0,1,1,1],
 [1,1,1,1,1],
 [1,0,0,1,0]
```

]

We build histograms row by row:

Row 0 → `[1,0,1,0,0]`
Row 1 → `[2,0,2,1,1]`
Row 2 → `[3,1,3,2,2]`
Row 3 → `[4,0,0,3,0]`

The largest rectangle appears in row 2 with height 2 and width 3, so
Area = 2 × 3 = 6

Approach 1

Algorithm

1. Convert the matrix into histogram heights:
    - If `matrix[i][j] == 1`, increase height from above
    - If `matrix[i][j] == 0`, reset height to `0`
2. For each row's histogram:
    - Use a stack-based method to find the largest rectangle in a histogram
3. Track the maximum area across all rows
4. Return the maximum area

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
private:
    int largestRectangleArea(vector<int> &heights) {
        int n = heights.size();
        stack<int> st;
        int maxArea = 0;

        for(int i = 0; i <= n; i++) {
            while(!st.empty() && (i == n ||
heights[st.top()] >= heights[i])) {
                int h = heights[st.top()];
                st.pop();
                int width = st.empty() ? i : i - st.top() -
1;
                maxArea = max(maxArea, h * width);
            }
            st.push(i);
        }
        return maxArea;
    }

public:
    int maximalAreaOfSubMatrixOfAll1(vector<vector<int>>
&matrix) {
        int n = matrix.size();
        int m = matrix[0].size();
```

```
        vector<vector<int>> prefix(n, vector<int>(m, 0));

        for(int j = 0; j < m; j++) {
            int height = 0;
            for(int i = 0; i < n; i++) {
                if(matrix[i][j] == 1) {
                    height++;
                } else {
                    height = 0;
                }
                prefix[i][j] = height;
            }
        }

        int maxArea = 0;
        for(int i = 0; i < n; i++) {
            maxArea = max(maxArea,
largestRectangleArea(prefix[i]));
        }

        return maxArea;
    }
};

int main() {
    vector<vector<int>> matrix = {
        {1,0,1,0,0},
        {1,0,1,1,1},
        {1,1,1,1,1},
        {1,0,0,1,0}
    };
```

```
    Solution sol;
    cout << sol.maximalAreaOfSubMatrixOfAll1(matrix);
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** `O(N × M)`
  Each row is processed as a histogram in linear time.

- **Space Complexity:** `O(N × M)`
  Prefix matrix plus stack space for histogram calculation.

27. Sliding Window Maximum

You are given an integer array `arr` and an integer `k`.
A sliding window of size `k` moves from the left of the array to the right, one step at a time.
For each window position, you must find the **maximum element** inside that window and return all such maximums.

A sliding window always contains **k consecutive elements**.

Question Explanation

At each step, the window includes `k` elements.
When the window moves right by one index:

- One element goes out of the window

- One new element comes into the window

For every such window, we only care about the **largest value** inside it.

Example Explanation

Input

`arr = [4,0,-1,3,5,3,6,8], k = 3`

Windows and maximums:

- `[4, 0, -1]` → max = 4
- `[0, -1, 3]` → max = 3
- `[-1, 3, 5]` → max = 5
- `[3, 5, 3]` → max = 5
- `[5, 3, 6]` → max = 6
- `[3, 6, 8]` → max = 8

Output

`[4, 3, 5, 5, 6, 8]`

Approach 1 (Brute Force)

Algorithm

- Loop through the array from index `0` to `n - k`
- For each position, consider the subarray of size k
- Scan all k elements to find the maximum
- Store the maximum in the result array
- Return the result

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    vector<int> maxSlidingWindow(vector<int>& nums, int k) {
        vector<int> result;
        int n = nums.size();

        for(int i = 0; i <= n - k; i++) {
            int maxi = nums[i];
            for(int j = i; j < i + k; j++) {
                maxi = max(maxi, nums[j]);
            }
            result.push_back(maxi);
        }
        return result;
    }
};
```

```
int main() {
    vector<int> arr = {4,0,-1,3,5,3,6,8};
    int k = 3;

    Solution sol;
    vector<int> ans = sol.maxSlidingWindow(arr, k);

    for(int x : ans) cout << x << " ";
    return 0;
}
```

- **Time Complexity:** `O(N * K)`
  For each window, we scan k elements.

- **Space Complexity:** `O(1)`
  No extra data structures used (output array excluded).

Approach 2 (Optimal – Deque)

Algorithm

- Use a deque to store **indices** of useful elements
- Maintain elements in decreasing order inside the deque
- For each index `i`:
    - Remove indices from the front if they are outside the window

- Remove indices from the back whose values are smaller than `arr[i]`
- Push current index to the back
- If window size is at least k, add `arr[dq.front()]` to result

- Return the result

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    vector<int> maxSlidingWindow(vector<int>& nums, int k) {
        deque<int> dq;
        vector<int> result;

        for(int i = 0; i < nums.size(); i++) {
            if(!dq.empty() && dq.front() <= i - k) {
                dq.pop_front();
            }

            while(!dq.empty() && nums[dq.back()] < nums[i])
{
                dq.pop_back();
            }

            dq.push_back(i);
```

```
            if(i >= k - 1) {
                result.push_back(nums[dq.front()]);
            }
        }
        return result;
    }
};

int main() {
    vector<int> arr = {4,0,-1,3,5,3,6,8};
    int k = 3;

    Solution sol;
    vector<int> ans = sol.maxSlidingWindow(arr, k);

    for(int x : ans) cout << x << " ";
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** `O(N)`
  Each element is inserted and removed from the deque at most once.

- **Space Complexity:** `O(K)`
  Deque stores at most k indices

28. Stock Span Problem

You are given an array `arr` of size `n`, where `arr[i]` represents the stock price on day `i`.
For each day, you need to calculate the **stock span**.

The **span** of a stock on day `i` is the maximum number of consecutive days **just before day `i` (including day `i`)** for which the stock price was **less than or equal to** the price on day `i`.

aaj se peechhe kitne continuous din tak stock price ≤ aaj ka price

---

For every day, we look backwards and count how many continuous days (including today) have prices **≤ today's price**.
The moment we find a price **greater than today's price**, we stop counting.

Example:

Input

```
arr = [120, 100, 60, 80, 90, 110, 115]
```

- Day 0 (120): no previous days → span = 1
- Day 1 (100): 100 < 120 → span = 1
- Day 2 (60): 60 < 100 → span = 1
- Day 3 (80): 80 ≥ 60, but < 100 → span = 2
- Day 4 (90): 90 ≥ 60, 80, but < 100 → span = 3

- Day 5 (110): 110 ≥ 60, 80, 90, 100 → span = 5
- Day 6 (115): 115 ≥ all except 120 → span = 6

Output

```
1  1  1  2  3  5  6
```

Approach 1 (Brute Force)

Algorithm

- Create an answer array of size n
- For each index `i`:
    - Start from `i` and move backwards
    - Count consecutive days where `arr[j] <= arr[i]`
    - Stop when a greater element is found
- Store the count as the span for that day
- Return the answer array

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
```

```
public:
    vector<int> stockSpan(vector<int> arr, int n) {
        vector<int> ans(n);

        for(int i = 0; i < n; i++) {
            int span = 0;
            for(int j = i; j >= 0; j--) {
                if(arr[j] <= arr[i])
                    span++;
                else
                    break;
            }
            ans[i] = span;
        }
        return ans;
    }
};

int main() {
    vector<int> arr = {120, 100, 60, 80, 90, 110, 115};
    int n = arr.size();

    Solution sol;
    vector<int> ans = sol.stockSpan(arr, n);

    for(int x : ans) cout << x << " ";
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** $O(N^2)$
  For each day, we may scan all previous days.

- **Space Complexity:** $O(1)$
  Only variables used (output array excluded).

Approach 2 (Optimal Using Stack)

Algorithm

- Use a stack to store **indices** of days with prices in decreasing order
- Traverse the array from left to right
- For each day:
  - Pop indices from stack while their price ≤ current price
  - If stack becomes empty → no previous greater element → span = `i + 1`
  - Else → span = `i - index_of_previous_greater`
- Push current index into the stack
- Return the span array

Jab tak peechhe ka price chhota ya barabar hai, wo aaj ke span ka hissa hai — use hata do.

Code

```cpp
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    vector<int> stockSpan(vector<int> arr, int n) {
        vector<int> ans(n);
        stack<int> st;

        for(int i = 0; i < n; i++) {
            while(!st.empty() && arr[st.top()] <= arr[i]) {
                st.pop();
            }

            if(st.empty())
                ans[i] = i + 1;
            else
                ans[i] = i - st.top();

            st.push(i);
        }
        return ans;
    }
};

int main() {
    vector<int> arr = {120, 100, 60, 80, 90, 110, 115};
    int n = arr.size();

    Solution sol;
    vector<int> ans = sol.stockSpan(arr, n);
```

```
    for(int x : ans) cout << x << " ";
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** `O(N)`
  Each element is pushed and popped from the stack at most once.

- **Space Complexity:** `O(N)`
  Stack can store up to `N` indices in the worst case.

29. Celebrity Problem

A celebrity is someone who **is known by everyone else** but **does not know anyone**.
You are given a square matrix `M` of size `N x N` where:

- `M[i][j] = 1` means person `i` knows person `j`
- `M[i][j] = 0` means person `i` does not know person `j`
- `M[i][i] = 0` for all `i`

Your task is to determine whether a celebrity exists.
If yes, return the **index** of the celebrity, otherwise return `-1`.

Example Explanation

Example 1

```
M = [
 [0,1,1,0],
 [0,0,0,0],
 [1,1,0,0],
 [0,1,1,0]
]
```

- Person 1 knows nobody (row 1 is all 0)
- Everyone else knows person 1
  So, person 1 is the celebrity.

Output: 1

Approach 1 (Brute Force)

Algorithm

- Create two arrays:
  - `knowMe[i]` → how many people know person `i`
  - `IKnow[i]` → how many people person `i` knows
- Traverse the entire matrix:

    - If `M[i][j] == 1`, increment `IKnow[i]` and `knowMe[j]`

- A person `i` is a celebrity if:
    - `knowMe[i] == N - 1`
    - `IKnow[i] == 0`
- If such a person exists, return their index, otherwise return `-1`

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int celebrity(vector<vector<int>> &M) {
        int n = M.size();
        vector<int> knowMe(n, 0), IKnow(n, 0);

        for(int i = 0; i < n; i++) {
            for(int j = 0; j < n; j++) {
                if(M[i][j] == 1) {
                    IKnow[i]++;
                    knowMe[j]++;
                }
            }
        }

        for(int i = 0; i < n; i++) {
```

```
            if(knowMe[i] == n - 1 && IKnow[i] == 0)
                return i;
        }
        return -1;
    }
};

int main() {
    vector<vector<int>> M = {
        {0,1,1,0},
        {0,0,0,0},
        {1,1,0,0},
        {0,1,1,0}
    };

    Solution sol;
    cout << sol.celebrity(M);
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** $O(N^2)$
  Full traversal of the matrix.

- **Space Complexity:** $O(N)$
  Two auxiliary arrays of size N.

Approach 2 (Optimal – Two Pointer)

Algorithm

- Use two pointers:
    - `top = 0`
    - `down = N - 1`
- While `top < down`:
    - If `top` knows `down`, then `top` cannot be celebrity → `top++`
    - Else if `down` knows `top`, then `down` cannot be celebrity → `down--`
    - Else both cannot be celebrity → move both
- After loop, `top` is the potential celebrity
- Verify:
    - Candidate knows nobody
    - Everyone knows the candidate
- If valid, return index, else return `-1`

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
```

```
    int celebrity(vector<vector<int>> &M) {
        int n = M.size();
        int top = 0, down = n - 1;

        while(top < down) {
            if(M[top][down] == 1)
                top++;
            else if(M[down][top] == 1)
                down--;
            else {
                top++;
                down--;
            }
        }

        for(int i = 0; i < n; i++) {
            if(i != top && (M[top][i] == 1 || M[i][top] ==
0))
                return -1;
        }
        return top;
    }
};

int main() {
    vector<vector<int>> M = {
        {0,1,1,0},
        {0,0,0,0},
        {1,1,0,0},
        {0,1,1,0}
    };
```

```
    Solution sol;
    cout << sol.celebrity(M);
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** `O(N)`
  One pass to eliminate candidates and one pass to verify.

- **Space Complexity:** `O(1)`
  Only constant extra variables used.

30. Implement LRU Cache

An **LRU (Least Recently Used) Cache** stores a fixed number of key–value pairs.
When the cache is full and a new key is inserted, the **least recently used** key is removed.

The challenge is that both operations must run in **O(1)** average time:

- `get(key)`

- `put(key, value)`

Key Idea

To achieve **O(1)** time for both operations, we combine **two data structures**:

1. **HashMap**
    - Maps `key → node`
    - Allows instant lookup
2. **Doubly Linked List**
    - Maintains usage order
    - Most Recently Used (MRU) → near head
    - Least Recently Used (LRU) → near tail

Why this works

- HashMap gives fast access
- Doubly linked list gives fast insert/delete
- When a key is accessed or updated → move it to the front
- When capacity exceeds → remove the node at the tail

Algorithm

Initialization

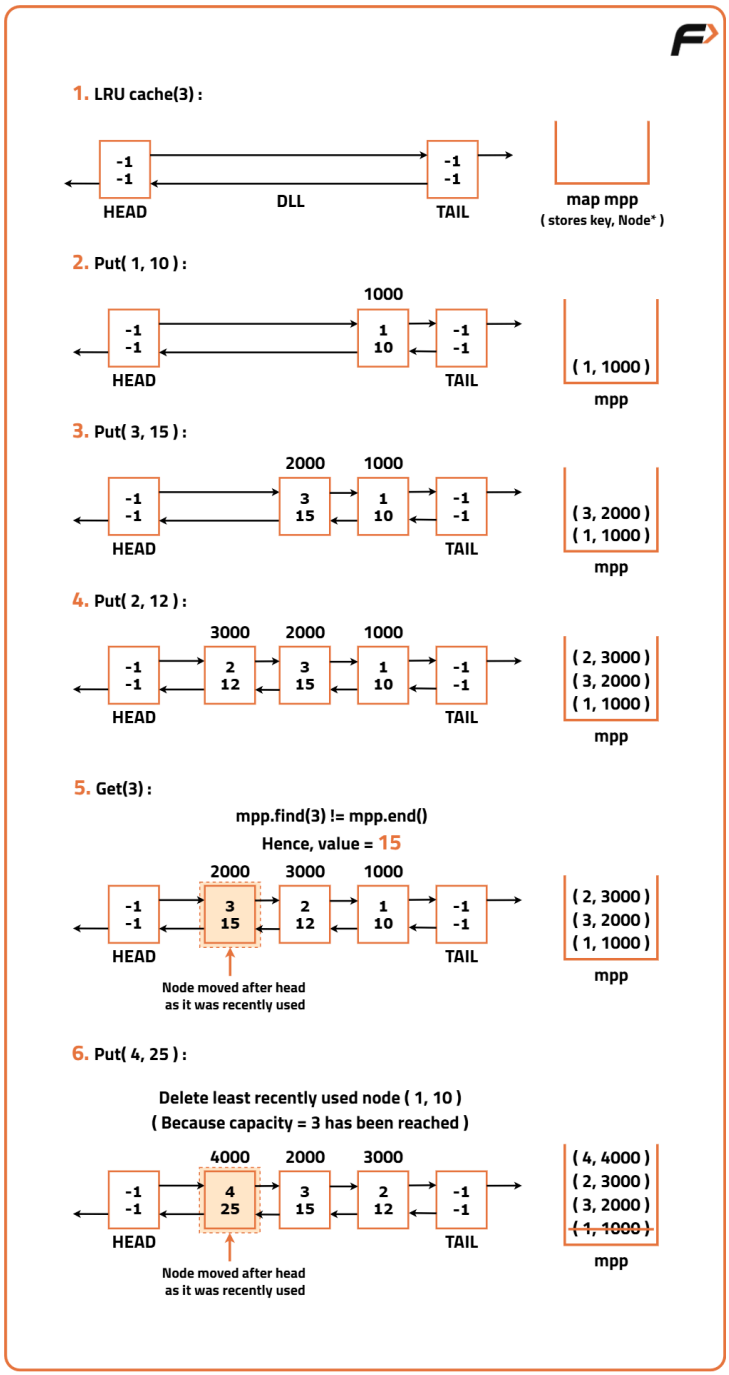
- Create dummy `head` and `tail` nodes
- Cache capacity is fixed

`get(key)`

- If key does not exist → return `-1`
- Else:
    - Move the node to the front (MRU)
    - Return its value

`put(key, value)`

- If key already exists:
    - Remove old node
- If cache is full:
    - Remove LRU node (node before tail)
- Insert new node at the front



C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;
```

```
class LRUCache {
public:
    class Node {
    public:
        int key, value;
        Node *prev, *next;
        Node(int k, int v) {
            key = k;
            value = v;
            prev = next = nullptr;
        }
    };

    int capacity;
    unordered_map<int, Node*> mp;
    Node *head, *tail;

    LRUCache(int cap) {
        capacity = cap;
        head = new Node(-1, -1);
        tail = new Node(-1, -1);
        head->next = tail;
        tail->prev = head;
    }

    void addNode(Node* node) {
        Node* temp = head->next;
        head->next = node;
        node->prev = head;
        node->next = temp;
        temp->prev = node;
```

```
}

void removeNode(Node* node) {
    Node* p = node->prev;
    Node* n = node->next;
    p->next = n;
    n->prev = p;
}

int get(int key) {
    if (mp.find(key) == mp.end())
        return -1;

    Node* node = mp[key];
    removeNode(node);
    addNode(node);
    return node->value;
}

void put(int key, int value) {
    if (mp.find(key) != mp.end()) {
        Node* node = mp[key];
        mp.erase(key);
        removeNode(node);
    }

    if (mp.size() == capacity) {
        Node* lru = tail->prev;
        mp.erase(lru->key);
        removeNode(lru);
    }
```

```
        Node* node = new Node(key, value);
        addNode(node);
        mp[key] = node;
    }
};

// Driver code
int main() {
    LRUCache cache(2);

    cache.put(1, 1);
    cache.put(2, 2);
    cout << cache.get(1) << endl; // 1

    cache.put(3, 3);
    cout << cache.get(2) << endl; // -1

    cache.put(4, 4);
    cout << cache.get(1) << endl; // -1
    cout << cache.get(3) << endl; // 3
    cout << cache.get(4) << endl; // 4

    return 0;
}
```

- **Time Complexity**
    - `get()` → **O(1)**
    - `put()` → **O(1)**

- **Space Complexity**
    - **O(capacity)**
      (HashMap + Doubly Linked List)

31. LFU Cache

You have to design a **Least Frequently Used (LFU) Cache**.
The cache stores key–value pairs and has a fixed capacity.

Rules of eviction:

- The key with the **lowest frequency of usage** is removed first.
- If multiple keys have the same frequency, the **least recently used (LRU)** key among them is removed.
- Every `get` or `put` on an existing key increases its frequency by 1.
- A newly inserted key starts with frequency = 1.
- Both `get` and `put` must work in **O(1) average time**.

Example Explanation

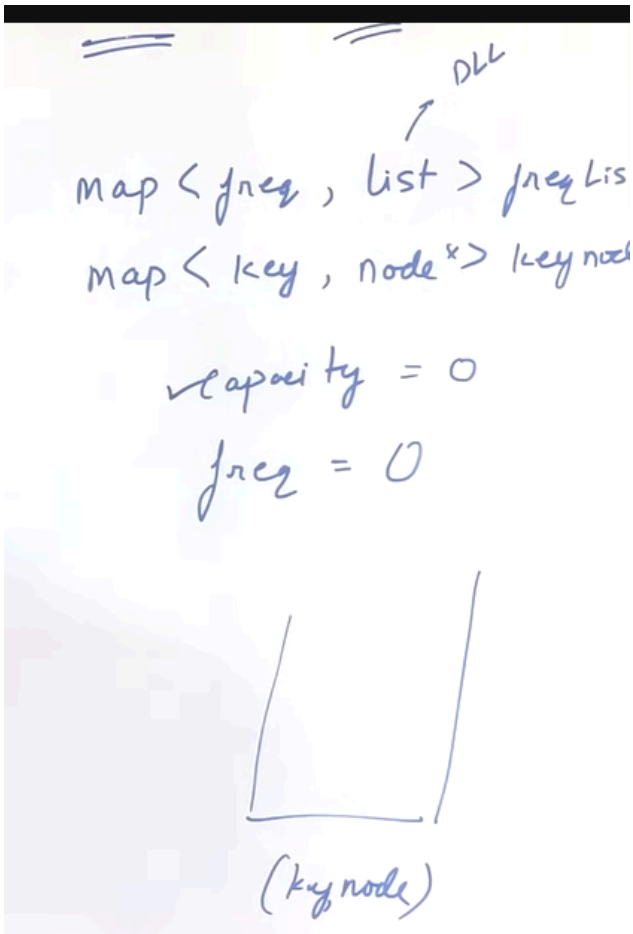
Capacity = 2

Operations:

- `put(1,1)` → cache: {1}, freq(1)=1
- `put(2,2)` → cache: {1,2}, freq both =1
- `get(1)` → returns 1, freq(1)=2
- `put(3,3)` → cache full, key 2 has lowest freq → evicted
- cache becomes {1,3}

This shows:

- LFU key is removed first
- If frequency ties, LRU is removed



Approach

Algorithm

- Use a **Node** to store `key`, `value`, and `frequency`.
- Use a **doubly linked list** for each frequency to maintain LRU order.
- Maintain:
    - `keyNode` → maps key to its node
    - `freqListMap` → maps frequency to its linked list
- Track:
    - `minFreq` → minimum frequency in cache
    - `curSize` → current cache size

For get(key):

- If key not found, return `-1`
- Else:
    - Increase its frequency
    - Move it to the next frequency list
    - Update `minFreq` if needed

For put(key, value):

- If capacity is `0`, do nothing
- If key exists:
    - Update value
    - Increase frequency
- Else:
    - If cache is full:
        - Remove LRU node from `minFreq` list
    - Insert new node with frequency = 1
    - Update `minFreq = 1`

Code

```
#include <bits/stdc++.h>
using namespace std;

struct Node {
    int key, value, cnt;
    Node *prev, *next;
    Node(int k, int v) {
        key = k;
        value = v;
        cnt = 1;
        prev = next = NULL;
    }
};
```

```
struct List {
    int size;
    Node *head, *tail;
    List() {
        head = new Node(0,0);
        tail = new Node(0,0);
        head->next = tail;
        tail->prev = head;
        size = 0;
    }
    void addFront(Node* node) {
        Node* temp = head->next;
        node->next = temp;
        node->prev = head;
        head->next = node;
        temp->prev = node;
        size++;
    }
    void removeNode(Node* node) {
        node->prev->next = node->next;
        node->next->prev = node->prev;
        size--;
    }
};

class LFUCache {
    int maxSize, curSize, minFreq;
    unordered_map<int, Node*> keyNode;
    unordered_map<int, List*> freqListMap;

public:
```

```
    LFUCache(int capacity) {
        maxSize = capacity;
        curSize = 0;
        minFreq = 0;
    }

    void updateFreq(Node* node) {
        keyNode.erase(node->key);
        freqListMap[node->cnt]->removeNode(node);

        if(node->cnt == minFreq &&
freqListMap[node->cnt]->size == 0)
            minFreq++;

        node->cnt++;
        if(freqListMap.find(node->cnt) == freqListMap.end())
            freqListMap[node->cnt] = new List();

        freqListMap[node->cnt]->addFront(node);
        keyNode[node->key] = node;
    }

    int get(int key) {
        if(keyNode.find(key) == keyNode.end())
            return -1;

        Node* node = keyNode[key];
        int val = node->value;
        updateFreq(node);
        return val;
    }
```

```
    void put(int key, int value) {
        if(maxSize == 0) return;

        if(keyNode.find(key) != keyNode.end()) {
            Node* node = keyNode[key];
            node->value = value;
            updateFreq(node);
        } else {
            if(curSize == maxSize) {
                List* list = freqListMap[minFreq];
                keyNode.erase(list->tail->prev->key);
                list->removeNode(list->tail->prev);
                curSize--;
            }

            curSize++;
            minFreq = 1;
            Node* node = new Node(key, value);

            if(freqListMap.find(minFreq) ==
freqListMap.end())
                freqListMap[minFreq] = new List();

            freqListMap[minFreq]->addFront(node);
            keyNode[key] = node;
        }
    }
};

int main() {
    LFUCache cache(2);
    cache.put(1,1);
```

```
    cache.put(2,2);
    cout << cache.get(1) << " ";
    cache.put(3,3);
    cout << cache.get(2) << " ";
    cout << cache.get(3) << " ";
    cache.put(4,4);
    cout << cache.get(1) << " ";
    cout << cache.get(3) << " ";
    cout << cache.get(4);
    return 0;
}
```

Complexity Analysis

- **Time Complexity:**
  `get` → O(1) average
  `put` → O(1) average

- **Space Complexity:**
  O(capacity), as the cache stores at most `capacity` elements